

随机数据,即使用原始的“暴力算法”,也能很快出解。如果你是命题者,可以这样出一个数据:有一个点很远,而其他 $n-1$ 个点相互比较近。这样,相距较近的 $n-1$ 个点之间的 $C(n-1,2)$ 条边会排序在前面,每次 Kruskal 都会先考虑完所有这些边。而考虑这些边时是无法让远点和近点连通的。

11.3 最短路问题

最短路问题并不陌生:在第 9 章中,曾介绍过无权和带权 DAG 上的最短路和最长路,二者的算法几乎是一样的(只是初始化不同,并且状态转移时把 min 和 max 互换)。但如果图中可以有环,情况就不同了。

11.3.1 Dijkstra 算法

Dijkstra 算法适用于边权为正的情况。下面直接给出 Dijkstra 算法的伪代码,它可用于计算正权图上的单源最短路(Single-Source Shortest Paths, SSSP),即从单个源点出发,到所有结点的最短路。该算法同时适用于有向图和无向图。

清除所有点的标号

设 $d[0]=0$, 其他 $d[i]=\text{INF}$

循环 n 次 {

 在所有未标号结点中,选出 d 值最小的结点 x

 给结点 x 标记

 对于从 x 出发的所有边 (x,y) ,更新 $d[y] = \min\{d[y], d[x]+w(x,y)\}$

}

下面是伪代码对应的程序。假设起点是结点 0,它到结点 i 的路径长度为 $d[i]$ 。未标号结点的 $v[i]=0$,已标号结点的 $v[i]=1$ 。为了简单起见,用 $w[x][y]=\text{INF}$ 表示边 (x,y) 不存在。

```
memset(v, 0, sizeof(v));
```

```
for(int i = 0; i < n; i++) d[i] = (i==0 ? 0 : INF);
```

```
for(int i = 0; i < n; i++) {
```

```
    int x, m = INF;
```

```
    for(int y = 0; y < n; y++) if(!v[y] && d[y]<=m) m = d[x=y];
```

```
    v[x] = 1;
```

```
    for(int y = 0; y < n; y++) d[y] = min(d[y], d[x] + w[x][y]);
```

```
}
```

除了求出最短路的长度外,使用 Dijkstra 算法也能很方便地打印出结点 0 到所有结点的最短路本身,原理和动态规划中的方案打印一样——从终点出发,不断顺着 $d[i]+w[i][j]=d[j]$ 的边 (i,j) 从结点 j “退回”到结点 i ,直到回到起点。另外,仍然可以用空间换时间,在更新 d 数组时维护“父亲指针”。具体来说,需要把 $d[y] = \min(d[y], d[x]+w[x][y])$ 改成:

```

if(d[y] > d[x] + w[x][y]) {
    d[y] = d[x] + w[x][y];
    fa[y] = x;
}

```

这称为边 (x,y) 上的松弛操作（relaxation）。不难看出，上面程序的时间复杂度为 $O(n^2)$ ——循环体一共执行了 n 次，而在每次循环中，“求最小 d 值”和“更新其他 d 值”均是 $O(n)$ 的。由于最短路算法实在太重要了，下面花一些篇幅把它优化到 $O(m\log n)$ ，并给出一份简单高效的完整代码。

等一等，为什么说是“优化到”呢？在最坏情况下， m 和 n^2 是同阶的， $m\log n$ 岂不是比 n^2 要大？这话没错，但在很多情况下，图中的边并没有那么多， $m\log n$ 比 n^2 小得多。 m 远小于 n^2 的图称为稀疏图（Sparse Graph），而 m 相对较大的图称为稠密图（Dense Graph）。

和前面一样，稀疏图适合使用 vector 数组保存。除此之外，还有一种流行的表示法——邻接表（Adjacency List）。在这种表示法中，每个结点 i 都有一个链表，里面保存着从 i 出发的所有边。对于无向图来说，每条边会在邻接表中出现两次。和前面一样，这里继续用数组实现链表：首先给每条边编号，然后用 $\text{first}[u]$ 保存结点 u 的第一条边的编号， $\text{next}[e]$ 表示编号为 e 的边的“下一条边”的编号。下面的函数读入有向图的边列表，并建立邻接表：

```

int n, m;
int first[maxn];
int u[maxm], v[maxm], w[maxm], next[maxm];
void read_graph() {
    scanf("%d%d", &n, &m);
    for(int i = 0; i < n; i++) first[i] = -1; //初始化表头
    for(int e = 0; e < m; e++) {
        scanf("%d%d%d", &u[e], &v[e], &w[e]);
        next[e] = first[u[e]]; //插入链表
        first[u[e]] = e;
    }
}

```

上述代码的巧妙之处是插入到链表的首部而非尾部，这样就避免了对链表的遍历。不过需要注意的是，同一个起点的各条边在邻接表中的顺序和读入顺序正好相反。读者如果还记得哈希表，应该会发现这里的链表和哈希表中的链表实现很相似。

尽管邻接表很流行，但在概念上 vector 数组更为简单，所以接下来仍然给出基于 vector 数组的代码。虽然在最短路问题中，每条边只有“边权”这一个属性，但后面的最大流以及最小费用流中还会出现“容量”、“流量”以及“费用”等属性。所以在这里使用一个称为 Edge 的结构体，这会让这里的代码与后面的代码在风格上更统一。

```

struct Edge {
    int from, to, dist;
    Edge(int u, int v, int d):from(u),to(v),dist(d) {}
};

```

为了使用方便, 此处把算法中用到的数据结构封装到一个结构体中:

```
struct Dijkstra {
    int n, m;
    vector<Edge> edges;
    vector<int> G[maxn];
    bool done[maxn];    //是否已永久标号
    int d[maxn];        //s 到各个点的距离
    int p[maxn];        //最短路中的上一条弧

    void init(int n) {
        this->n = n;
        for(int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int dist) {
        edges.push_back(Edge(from, to, dist));
        m = edges.size();
        G[from].push_back(m-1);
    }

    void dijkstra(int s) {
        ...
    }
};
```

不难看出, 在 `vector` 数组中保存的只是边的编号。有了编号之后可以从 `edges` 数组中查到边的具体信息。有了这样的数据结构, “遍历从 x 出发的所有边 (x,y) , 更新 $d[y]$ ” 就可以写成 “`for(int i = 0; i < G[u].size(); i++)` 执行边 `edges[G[u][i]]` 上的松弛操作”。尽管在最坏情况下, 这个循环仍然会循环 $n-1$ 次, 但从整体上来看, 每条边恰好被检查过一次 (想一想, 为什么), 因此松弛操作执行的次数恰好是 m 。这样, 只需集中精力优化 “找出未标号结点中的最小 d 值” 即可。

在 Dijkstra 算法中, $d[i]$ 越小, 应该越先出队, 因此需要使用自定义比较器。在 STL 中, 可以用 `greater<int>` 表示 “大于” 运算符, 因此可以用 `priority_queue<int, vector<int>, greater<int>>` 来声明一个小整数先出队的优先队列。然而, 除了需要最小的 d 值之外, 还要找到这个最小值对应的结点编号, 所以要把 d 值和编号 “捆绑” 成一个整体放到优先队列中, 使得取出最小 d 值的同时也会取出对应的结点编号。

STL 中的 `pair` 便是专门把两个类型捆绑到一起的。为了方便起见, 用 `typedef pair<int,int> pii` 自定义一个 `pii` 类型, 则 `priority_queue<pii, vector<pii>, greater<pii>>` 就定义了一个由二元组构成的优先队列。`pair` 定义了它自己的排序规则——先比较第一维, 相等时才比较第二维, 因此需要按 $(d[i], i)$ 而不是 $(i, d[i])$ 的方式组合。这样的方法理论上和实际上都没有问题,

很多用户并不习惯。为了保持简单，这里不使用 `pair`，而是显式定义一个结构体作为优先队列中的元素类型，例如：

```
struct HeapNode {
    int d, u;
    bool operator < (const HeapNode& rhs) const {
        return d > rhs.d;
    }
};
```

然后主算法就可以写出来了：

```
void dijkstra(int s) {
    priority_queue<HeapNode> Q;
    for(int i = 0; i < n; i++) d[i] = INF;
    d[s] = 0;
    memset(done, 0, sizeof(done));
    Q.push((HeapNode){0, s});
    while(!Q.empty()) {
        HeapNode x = Q.top(); Q.pop();
        int u = x.u;
        if(done[u]) continue;
        done[u] = true;
        for(int i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]];
            if(d[e.to] > d[u] + e.dist) {
                d[e.to] = d[u] + e.dist;
                p[e.to] = G[u][i];
                Q.push((HeapNode){d[e.to], e.to});
            }
        }
    }
}
```

在松弛成功后，需要修改结点 `e.to` 的优先级，但 STL 中的优先队列不提供“修改优先级”的操作。因此，只能将新元素重新插入优先队列。这样做并不会影响结果的正确性，因为 `d` 值小的结点自然会先出队。为了防止结点的重复扩展，如果发现新取出来的结点曾经被取出来过（`done[u]`），应该直接把它扔掉。避免重复的另一个方法是把 `if(done[u])` 改成 `if(x.d != d[u])`，可以省掉一个 `done` 数组。

再补充一点：即使是稠密图，使用 `priority_queue` 实现的 Dijkstra 算法也常常比基于邻接矩阵的 Dijkstra 算法的运算速度快。理由很简单，执行 `push` 操作的前提是 `d[e.to] > d[u] + e.dist`，如果这个式子常常不成立，则 `push` 操作会很少。

11.3.2 Bellman-Ford 算法

当负权存在时，连最短路都不一定存在了。尽管如此，还是有办法在最短路存在的情况下把它求出来。在介绍算法之前，请读者确认这样一个事实：如果最短路存在，一定存在一个不含环的最短路。

理由如下：在边权可正可负的图中，环有零环、正环和负环 3 种。如果包含零环或正环，去掉以后路径不会变长；如果包含负环，则意味着最短路不存在（想一想，为什么）。

既然不含环，最短路最多只经过（起点不算） $n-1$ 个结点，可以通过 $n-1$ “轮” 松弛操作得到，像这样（起点仍然是 0）：

```
for(int i = 0; i < n; i++) d[i] = INF;
d[0] = 0;
for(int k = 0; k < n-1; k++) //迭代 n-1 次
    for(int i = 0; i < m; i++) //检查每条边
    {
        int x = u[i], y = v[i];
        if(d[x] < INF) d[y] = min(d[y], d[x]+w[i]); //松弛
    }
```

上述算法称为 Bellman-Ford 算法，不难看出它的时间复杂度为 $O(nm)$ 。在实践中，常常用 FIFO 队列来代替上面的循环检查，像这样：

```
bool bellman_ford(int s) {
    queue<int> Q;
    memset(inq, 0, sizeof(inq));
    memset(cnt, 0, sizeof(cnt));
    for(int i = 0; i < n; i++) d[i] = INF;
    d[s] = 0;
    inq[s] = true;
    Q.push(s);

    while(!Q.empty()) {
        int u = Q.front(); Q.pop();
        inq[u] = false;
        for(int i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]];
            if(d[u] < INF && d[e.to] > d[u] + e.dist) {
                d[e.to] = d[u] + e.dist;
                p[e.to] = G[u][i];
                if(!inq[e.to]) { Q.push(e.to); inq[e.to] = true; if(++cnt[e.to] >
n) return false; }
            }
        }
    }
```

```

    }
  }
  return true;
}

```

有没有注意到上面的代码和前面的 Dijkstra 算法很像？一方面，优先队列替换为了普通的 FIFO 队列，而另一方面，一个结点可以多次进入队列。可以证明，采取 FIFO 队列的 Bellman-Ford 算法在最坏情况下需要 $O(nm)$ 时间，不过在实践中，往往只需要很短的时间就能求出最短路。上面的代码还有一个功能：在发现负圈时及时退出。注意，这只说明 s 可以到达一个负圈，并不代表 s 到每个点的最短路都不存在。另外，如果图中有其他负圈但是 s 无法到达这个负圈，则上面的算法也无法找到。解决方法留给读者思考（提示：加一个结点）。

11.3.3 Floyd 算法

如果要求出每两点之间的最短路，不必调用 n 次 Dijkstra（边权均为正）或者 Bellman-ford（有负权）。有一个更简单的方法可以实现——Floyd-Warshall 算法（请记住下面的代码！）：

```

for(int k = 0; k < n; k++)
  for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
      d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

```

在调用它之前只需做一些简单的初始化： $d[i][i]=0$ ，其他 d 值为“正无穷” INF 。注意这里有一个潜在的问题：如果 INF 定义太大（如 2000000000），加法 $d[i][k] + d[k][j]$ 可能会溢出！但如果 INF 太小，可能会使得长度为 INF 的边真的变成最短路的一部分。谨慎起见，最好估计一下实际最短路长度的上限，并把 INF 设置成“只比它大一点点”的值。例如，最多有 1000 条边，若每条边长度不超过 1000，可以把 INF 设成 1000001。

如果坚持认为不应该允许 INF 和其他值相加，更不应该得到一个大于 INF 的数，请把上述代码改成：

```

for(int k = 0; k < n; k++)
  for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
      if(d[i][j] < INF && d[k][j] < INF)
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

```

在有向图中，有时不必关心路径的长度，而只关心每两点间是否有通路，则可以用 1 和 0 分别表示“连通”和“不连通”。这样，除了预处理需做少许调整外，主算法中只需把“ $d[i][j] = \min\{d[i][j], d[i][k] + d[k][j]\}$ ”改成“ $d[i][j] = d[i][j] \vee (d[i][k] \wedge d[k][j])$ ”。这样的结果称为有向图的传递闭包（Transitive Closure）。

11.3.4 竞赛题目选讲

例题 11-4 电话圈 (Calling Circles, ACM/ICPC World Finals 1996, UVa247)

如果两个人相互打电话（直接或间接），则说他们在同一个电话圈里。例如，a 打给 b，b 打给 c，c 打给 d，d 打给 a，则这 4 个人在同一个圈里；如果 e 打给 f 但 f 不打给 e，则不能推出 e 和 f 在同一个电话圈里。输入 n ($n \leq 25$) 个人的 m 次电话，找出所有电话圈。人名只包含字母，不超过 25 个字符，且不重复。

【分析】

首先用 floyd 求出传递闭包，即 $g[i][j]$ 表示 i 是否直接或者间接给 j 打过电话，则当且仅当 $g[i][j]=g[j][i]=1$ 时二者处于一个电话圈。构造一个新图，在“在一个电话圈里”的两个人之间连一条边，然后依次输出各个连通分量的所有人即可。

例题 11-5 噪音恐惧症 (Audiophobia, UVa10048)

输入一个 C 个点 S 条边 ($C \leq 100, S \leq 1000$) 的无向带权图，边权表示该路径上的噪声值。当噪声值太大时，耳膜可能会受到伤害，所以当你从某点去往另一个点时，总是希望路上经过的最大噪声值最小。输入一些询问，每次询问两个点，输出这两点间最大噪声值最小的路径。例如，在图 11-8 中，A 到 G 的最大噪声值为 80，是所有其他路径中最小的（如 ABEG 的最大噪声值为 90）。

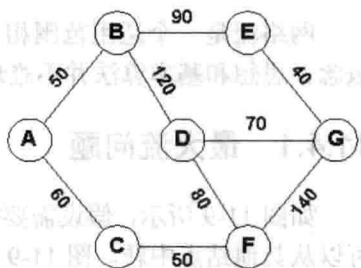


图 11-8 路径与噪声值

【分析】

本题的做法十分简单：直接用 floyd 算法，但是要把加法改成 min，min 改成 max。为什么可以这样做呢？不管是 floyd 算法还是 dijkstra 算法，都是基于这样一个事实：对于任意一条至少包含两条边的路径 $i \rightarrow j$ ，一定存在一个中间点 k ，使得 $i \rightarrow j$ 的总长度等于 $i \rightarrow k$ 与 $k \rightarrow j$ 的长度之和。对于不同的点 k ， $i \rightarrow k$ 和 $k \rightarrow j$ 的长度之和可能不同，最后还需要取一个最小值才是 $i \rightarrow j$ 的最短路径。把刚才的推理中“之和”与“取最小值”换成“取最小值”和“取最大值”，推理仍然适用。

例题 11-6 这不是 bug，而是特性 (It's not a Bug, it's a Feature!, UVa 658)

补丁在修正 bug 时，有时也会引入新的 bug。假定有 n ($n \leq 20$) 个潜在 bug 和 m ($m \leq 100$) 个补丁，每个补丁用两个长度为 n 的字符串表示，其中字符串的每个位置表示一个 bug。第一个串表示打补丁之前的状态（“-”表示该 bug 必须不存在，“+”表示必须存在，0 表示无所谓），第二个串表示打补丁之后的状态（“-”表示不存在，“+”表示存在，0 表示不变）。每个补丁都有一个执行时间，你的任务是用最少的时间把一个所有 bug 都存在的软件通过打补丁的方式变得没有 bug。一个补丁可以打多次。

【分析】

在任意时刻，每个 bug 可能存在也可能不存在，所以可以用一个 n 位二进制串表示当前软件的“状态”。打完补丁之后，bug 状态会发生改变，对应“状态转移”。是不是很像动态规划？可惜动态规划是行不通的，因为状态经过多次转移之后可能会回到以前的状态，